Mixed Criticality Scheduling

Nader Yahia
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
nader.yahia@stud.hshl.de

***Abstract*—Mixed-criticality scheduling addresses the challenge of executing tasks with different safety assurance requirements on a shared computing platform. Traditional real-time scheduling relies on a single Worst-Case Execution Time (WCET) assumption, which either leads to excessive resource reservation or insufficient guarantees for safety-critical tasks. Mixed-criticality scheduling resolves this limitation by assigning multiple WCET estimates according to different assurance levels and enabling mode-dependent execution. This paper reviews the fundamental concepts of mixed-criticality systems, including the task model, criticality levels, WCET assumptions, and operating-mode transitions. Furthermore, the Vestal model and EDF with Virtual Deadlines (EDF-VD) are analyzed as representative approaches for providing timing guarantees under execution-time uncertainty. The analysis highlights the inherent trade-off between safety guarantees and resource efficiency, where high-criticality tasks are protected at the cost of reduced service for low-criticality workloads. Finally, limitations related to WCET pessimism, low-criticality task degradation, and multicore scalability are discussed.**

I. Introduction

II. Mixed-Criticality Scheduling

A. Background

Real-time embedded systems are considered the backbone of most safety-critical infrastructure: avionics flight control, automotive powertrain management, medical infusion devices, and railway interlocking. Across all safety-critical domains, deadline compliance is equally significant to the correctness of logical outputs. It must be produced before hard deadlines, unconditionally and verifiably. Missing a deadline in these scenarios is not a performance degradation; it is a system-level failure with potentially serious, life-threatening consequences. The classical real-time scheduling theory introduced by Liu and Layland [1], provides rigorous schedulability conditions for periodic task sets. Within this framework, a single Worst-Case Execution Time (WCET) bound a C_i is assigned to each task i. and a system is deemed schedulable exactly when no task ever misses its deadline, assuming every task always executes for a duration of C_i time units.

B. Motivation

First, The Safety-Critical Co-Hosting Problem: Contemporary system integration is driven by size, weight, power, and cost (SWaP-C) constraints, leading to sharing a single processing platform among systems with different safety criticality levels. An avionics Integrated Modular Avionics (IMA) unit, for example, must simultaneously host flight-control software (DO-178C DAL-A) and cabin entertainment management (DAL-D) on the same processor [2]. Similarly, an automotive System-on-Chip (SoC) may execute both an ASIL-D braking control loop and an ASIL-A infotainment service concurrently. Under classical theory, this is handled either through strict temporal partitioning or by treating all co-hosted tasks as if they share the highest criticality level. The former wastes resources through rigid time allocation, while the latter imposes conservative WCET bounds on non-critical tasks, introducing significant pessimism and often rendering schedulability infeasible even on capable hardware.

Second, The WCET Uncertainty Problem: Real-time systems cannot rely on a single WCET value per task because execution-time analysis depends on the required level of timing assurance. High-criticality tasks are analyzed using conservative, certification-grade methods, resulting in safe but pessimistic WCET bounds, while low-criticality tasks typically use measurement-based estimates that are tighter but less reliable. This mismatch implies that a single WCET cannot consistently represent all tasks under different assurance levels. Consequently, mixed-criticality systems model each task with two execution-time bounds, $C_i(LO) \leq C_i(HI)$, forming the basis for mixed-criticality scheduling theory.

The safety-critical co-hosting problem and the WCET uncertainty problem expose fundamental limitations of traditional single-WCET scheduling models. To address these challenges, Vestal [3] proposed the mixed-criticality scheduling paradigm, in which tasks are associated with multiple execution-time assumptions corresponding to different assurance levels. The remainder of this paper is organized as follows. Section II presents the system model, including task criticality levels, execution-time assumptions, and operating modes. Section III describes the scheduling algorithm and mode-switching mechanisms. Section IV provides the corresponding schedulability analysis and discusses the resulting guarantees under mixed-criticality assumptions.

III. System Model

A. Task Model

The system consists of a set of n sporadic or periodic tasks $T = \{\tau_1, \tau_2, \ldots, \tau_n\}$. Each task τ_i is defined by the

tuple: $\tau_i = (T_i, D_i, C_i(LO), C_i(HI), L_i)$, where T_i denotes the task period (or minimum inter-arrival time), D_i represents the relative deadline, and $C_i(LO)$ and $C_i(HI)$ denote the WCET estimates under low- and high-criticality assurance levels, respectively. The parameter $L_i \in \{LO, HI\}$ defines the criticality level of task τ_i. The constrained deadline model is assumed throughout: $D_i \leq T_i \quad \forall \tau_i \in$ The implicit deadline model ($D_i = T_i$) is a special case. Arbitrary deadlines ($D_i > T_i$) are excluded from this foundational treatment.

B. Criticality Levels

The criticality level $L_i \in \{LO, HI\}$ represents the assurance level and consequence of a deadline failure. HI-criticality tasks are assigned two WCET estimates, $C_i(LO)$ and $C_i(HI)$, where $C_i(LO) \leq C_i(HI)$, while LO-criticality tasks use only $C_i(LO)$. Although the model can be extended to multiple criticality levels, this work adopts the binary LO/HI formulation [4].

C. Assumptions

The system model is based on standard real-time scheduling assumptions defining the execution environment of the proposed framework. These assumptions ensure analyzability while preserving relevance to embedded real-time systems; they are summarized in Table I.

Assumption	Name	Description
A1	Single-Core Processor	All tasks execute on a single processor.
A2	Preemptive Scheduling	Jobs may be preempted at any instant.
A3	Task Independence	Tasks are independent and do not share synchronization objects.
A4	Deterministic WCET Bounds	The values $C_i(LO)$ and $C_i(HI)$ are assumed correct.
A5	Periodic Activation	Tasks are released periodically with period T_i.
A6	No Resource Sharing	No semaphores, mutexes, or shared I/O resources are considered.

TABLE I
SYSTEM MODEL ASSUMPTIONS

D. WCET Bound Interpretation

For every job:

$$\forall k \in \mathbb{N} : e_k(\tau_i) \leq C_i(L_i)$$

The ratio

$$\frac{C_i(HI)}{C_i(LO)}$$

is commonly referred to as the criticality augmentation factor.

E. Schedulability Condition

A mixed-criticality task set satisfies the basic correctness conditions if: First, all tasks meet their deadlines in LO-mode under the assumption that execution does not exceed $C_i(LO)$. Second, all HI-criticality tasks meet their deadlines in HI-mode under the assumption that execution does not exceed $C_i(HI)$. LO-criticality task deadlines are not required to be guaranteed in HI-mode. This bimodal correctness requirement distinguishes mixed-criticality scheduling from classical single-mode real-time scheduling.

IV. ALGORITHMS AND MODE MANAGEMENT

A. Preliminaries: Fixed-Priority and EDF Scheduling

Two canonical approaches govern real-time task scheduling on uniprocessors: fixed-priority (FP) scheduling and dynamic-priority (EDF-based) scheduling. Under FP scheduling, each task τ_i is assigned a static priority π_i at design time; at runtime, the ready task with the highest priority always executes. Rate-Monotonic (RM) assignment ($\pi_i \propto 1/T_i$) is optimal among FP policies for implicit-deadline periodic tasks [1]. FP scheduling is simple, predictable, and hardware-friendly, but its utilization bound of $U \leq n(2^{1/n} - 1) \to \ln 2 \approx 0.693$ as $n \to \infty$ forms a hard ceiling.

Under Earliest Deadline First (EDF), the ready task with the earliest absolute deadline executes at all times. EDF is optimal on uniprocessors in the sense that if any scheduler can meet all deadlines for a feasible task set, EDF will also meet them [1]. It achieves full processor utilization ($U \leq 1$), but its dynamic priority assignment leads to less predictable behavior under overload conditions compared to FP.

In the mixed-criticality context, both paradigms have been extended. FP-based approaches (e.g., AMC — Adaptive Mixed-Criticality) assign criticality-aware priority levels based on task criticality, while EDF-based approaches extend EDF by modifying its scheduling policy to account for multiple execution-time assumptions. Among these, EDF-VD has emerged as one of the most widely studied and analytically tractable algorithms and is therefore examined in detail in the following sections.

B. The Vestal Model and Multi-WCET Scheduling

Vestal's foundational observation [3] directly motivates mixed-criticality scheduling by highlighting the inefficiency of using a single WCET estimate for all execution contexts. In such a case, the system must be dimensioned using the most pessimistic execution-time bound, which can lead to overly conservative resource allocation.

For a task set T with HI-criticality tasks satisfying $C_i(HI) \gg C_i(LO)$, the resulting utilization under a single-mode assumption is:

$$U_{\text{naive}} = \sum_{\tau_i \in T} \frac{C_i(HI)}{T_i}$$

which may render otherwise feasible systems infeasible under nominal operating conditions.

Vestal's approach: Instead of a single WCET value, each HI-criticality task is assigned two execution budgets, $C_i(LO)$ and $C_i(HI)$. The system executes optimistically using $C_i(LO)$ during normal operation. If any HI-criticality task exceeds its LO budget, a global mode switch is triggered, after which the system is scheduled using HI-mode budgets, and LO-criticality tasks may be suspended. This yields a two-tier correctness guarantee: **(i)** in LO-mode, all tasks must meet their deadlines assuming execution does not exceed $C_i(LO)$, and **(ii)** in HI-mode, all HI-criticality tasks must meet their deadlines assuming execution does not exceed $C_i(HI)$, even if LO-criticality tasks are dropped. A task set T is considered mixed-criticality feasible if both guarantees can be satisfied simultaneously under some scheduling policy.

C. EDF-Based Mixed-Criticality Scheduling

Under standard EDF applied to a mixed-criticality task set, each task τ_i is assigned an absolute deadline $d_{i,k} = r_{i,k} + D_i$ for its k-th job, where $r_{i,k}$ denotes the release time. The scheduler always executes the ready job with the earliest absolute deadline. If tasks are scheduled using their LO-mode execution budgets $C_i(\mathrm{LO})$, the system is schedulable in LO-mode whenever $U(\mathrm{LO}) \leq 1$. However, when a HI-criticality task executes beyond its LO-mode budget $C_i(\mathrm{LO})$, standard EDF has no mechanism to distinguish between task criticality levels. Consequently, a HI-criticality task may be preempted by a LO-criticality task with an earlier deadline, potentially causing the HI-criticality task to miss its deadline. This represents a correctness failure rather than merely a performance degradation. The fundamental challenge is therefore how to bias EDF in favor of HI-criticality tasks without knowing in advance which jobs will exceed their LO-mode execution budgets.

This limitation motivates EDF with Virtual Deadlines (EDF-VD), introduced by Baruah et al. [4]. The key idea is to assign HI-criticality tasks virtual deadlines that are earlier than their true deadlines. As a result, HI-criticality tasks receive additional execution time in LO-mode, creating scheduling headroom that can later be reclaimed if a mode switch occurs. Thus, when a HI-criticality task overruns its LO-mode budget and the system transitions to HI-mode, sufficient execution capacity remains before the task's true deadline to accommodate execution up to $C_i(\mathrm{HI})$.

For each task τ_i, EDF-VD defines an effective scheduling deadline $\hat{D}_i$ as

$$\hat{D}_i = \begin{cases} xD_i, & \text{if } L_i = HI,\ x \in (0,1], \\ D_i, & \text{if } L_i = LO, \end{cases}$$

where x is a compression factor applied uniformly to all HI-criticality tasks. During LO-mode, jobs are scheduled according to the virtual deadlines $\hat{d} * i, k = r * i, k + \hat{D} * i$. Following a mode switch, virtual deadlines are discarded and scheduling reverts to standard EDF using the true deadlines $d * i, k = r_{i,k} + D_i$.

The value of x determines how aggressively HI-criticality task deadlines are compressed during LO-mode operation. Smaller values of x provide greater scheduling preference to HI-criticality tasks, while larger values approach standard EDF behavior. The compression factor is computed offline to ensure that both LO-mode and HI-mode schedulability requirements can be satisfied. By reserving execution capacity for HI-criticality tasks before a mode switch occurs, EDF-VD provides stronger schedulability guarantees than naive EDF in mixed-criticality systems.

D. Mode Switching Logic

A transition from LO-mode to HI-mode occurs when a HI-criticality task exceeds its LO-mode execution budget during runtime. Upon detection, the system mode is changed to HI, LO-criticality tasks are suspended, and HI-criticality tasks continue using their true deadlines and higher WCET bounds. The transition operation must execute atomically with respect to the scheduler, ensuring that no task scheduling decision occurs during the mode change and preventing deadline violations caused by inconsistent system states.

Returning from HI-mode to LO-mode is not defined by the core EDF-VD theory and is treated as a system-level policy decision. Possible strategies include restoration after a hyperperiod, after completion of the overrunning task, or after a predefined number of overrun-free intervals. The selected strategy determines the trade-off between LO-task availability and safety guarantees.

V. Analysis

A. Formal Analysis

In classical real-time scheduling, schedulability is determined by analyzing the worst-case response time of each task. A task is considered schedulable if its maximum execution delay remains within its deadline. This analysis accounts for interference caused by higher-priority tasks competing for processor time. In mixed-criticality systems, response-time analysis is extended to consider multiple operating modes. During LO-mode operation, all tasks are analyzed using their optimistic WCET estimates, allowing both high- and low-criticality tasks to execute. If a high-criticality task exceeds its LO-mode execution budget, the system transitions to HI-mode. In this state, only high-criticality tasks are guaranteed execution, and the analysis uses their more conservative WCET bounds while low-criticality tasks may be suspended. Correctness therefore requires that all tasks satisfy their timing requirements in LO-mode and that all HI-criticality tasks remain schedulable after a mode transition. The transition overhead must also be bounded to ensure that switching between modes does not itself cause deadline violations.

Interference Reasoning Under Mode Duality,To capture the impact of mode-dependent execution assumptions in mixed-criticality systems, we define the mode-conditional interference contributed by task τ_j to task τ_i over an interval $[t, t+L]$ as:

$$I_{j\to i}(\ell, L) = \left\lceil \frac{L}{T_j} \right\rceil C_j(\ell) \tag{1}$$

where $\ell \in \{LO, HI\}$ denotes the current operating mode, T_j is the period of task τ_j, and $C_j(\ell)$ is the execution budget assumed under mode ℓ. In LO-mode ($\ell = LO$), all tasks in the system contribute interference using their LO-mode execution budgets $C_j(LO)$. In contrast, in HI-mode ($\ell = HI$), only HI-criticality tasks continue to execute, and therefore only tasks with $L_j = HI$ contribute interference, using their HI-mode execution budgets $C_j(HI)$. LO-criticality tasks are suspended and contribute no further interference. This creates a fundamental asymmetry between the two operating modes. In LO-mode, the system experiences maximal workload due to the simultaneous execution of all tasks. In HI-mode, the system intentionally reduces workload by disabling LO-criticality execution, thereby lowering interference on safety-critical tasks.

Without LO-task suspension, the HI-mode workload under worst-case assumptions would be given by:

$$U_{all}^{HI} = \sum_{\tau_i \in T} \frac{C_i(HI)}{T_i} \tag{2}$$

which may exceed unity and thus violate processor capacity constraints. With LO-task suspension, the effective HI-mode utilization becomes:

$$U_{reduced}^{HI} = \sum_{\tau_i \in T,\, L_i = HI} \frac{C_i(HI)}{T_i} \tag{3}$$

This reduction in interference is a key enabler of HI-mode schedulability. The mixed-criticality paradigm therefore relies on controlled degradation of LO-criticality workload as a mechanism to preserve timing guarantees for safety-critical tasks under overload conditions. Rather than being a disadvantage, LO-task suspension is a deliberate design choice that transforms potential overload situations into schedulable configurations for HI-criticality execution. This reflects a fundamental tradeoff between system efficiency in LO-mode and temporal guarantees in HI-mode.

B. Tradeoff Analysis

Safety vs. Efficiency, This is the foundational tension in mixed-criticality systems. Safety demands that HI-criticality tasks always meet their deadlines, regardless of execution conditions. Efficiency, in contrast, demands maximal utilization of processor resources, including execution of LO-criticality tasks. These objectives are structurally opposed. Maximizing HI-task safety requires reserving execution headroom corresponding to the difference between $C_i(LO)$ and $C_i(HI)$, ensuring that execution overruns remain schedulable at runtime. Maximizing efficiency requires exploiting this reserved capacity for LO-task execution. The EDF-VD mechanism introduces a compression factor x^* that operationalizes this tradeoff. Smaller values of x^* increase safety by providing larger HI-mode slack, while reducing available bandwidth for LO-task execution.

WCET Pessimism vs. Utilization, The correctness of mixed-criticality scheduling depends critically on WCET bounds. However, HI-mode execution budgets $C_i(HI)$ are inherently conservative over-approximations derived from worst-case hardware assumptions. In practice:

$$C_i(HI) \gg e_k(\tau_i) \tag{4}$$

The ratio $\rho_i = C_i(HI)/C_i(LO)$ quantifies the WCET inflation factor. In practice, this factor can become significantly larger than one because high-assurance WCET analysis introduces conservative assumptions to guarantee timing correctness under worst-case conditions [5]. This conservatism reduces processor utilization in HI-mode, as reserved computation capacity may remain unused during typical execution. This effect is commonly referred to as the *WCET pessimism tax*.

Key research tension: tightening WCET bounds improves utilization efficiency but risks unsafe underestimation, whereas conservative bounds ensure safety but degrade system efficiency. Resolving this tradeoff remains an open research challenge in WCET analysis and mixed-criticality scheduling theory.

LO-Task Degradation vs. HI-Task Guarantees, The mode-switch mechanism introduces a deliberate discontinuity in LO-task execution. Upon a HI-mode transition, LO-criticality tasks are suspended, potentially for extended durations. In the worst case, a mode switch at time t_s may persist until the end of the hyperperiod $H = \mathrm{lcm}(T_i)$, resulting in LO-task starvation intervals of length: $H - (t_s \bmod H)$. For task systems with large hyperperiods, this can result in substantial service interruptions for LO-criticality workloads. Define the LO availability ratio as:

$$\alpha_{LO} = \frac{\mathbb{E}[\text{time in LO-mode}]}{H} \tag{5}$$

Under nominal conditions, $\alpha_{LO} \approx 1$, while under repeated or adversarial overruns, $\alpha_{LO} \to 0$. Importantly, classical MCS theory does not provide a strict lower bound on α_{LO}, as LO-task service is not part of the formal guarantee model. This creates a fundamental asymmetry: HI-task guarantees are formally proven and certification-grade, whereas LO-task service quality is best-effort and only empirically bounded. For systems where LO tasks perform non-trivial soft real-time functions, this asymmetry represents a significant architectural limitation masked by theoretical guarantees on HI-criticality correctness.

C. Limitations

WCET Uncertainty, The entire MCS framework rests on the assumption that $C_i(HI)$ is a correct safe upper bound—that no job ever executes longer than $C_i(HI)$. This assumption is not self-verifying, It is:

- Hardware-platform specific: A WCET bound valid for a RISC-V in-order core (e.g., Ibex) is invalid for an out-of-order core with speculative execution or a multi-level cache hierarchy.
- Dependent on analysis completeness: Static WCET tools require manual loop bound annotations, memory layout information, and hardware timing models. Errors in any input produce an unsafe bound.
- Violated by hardware anomalies: Timing anomalies in pipelined processors—where a faster decision at one stage leads to a slower overall execution due to pipeline interactions—can make the sum of per-segment WCET bounds non-composable [5].
- Not certifiable for all hardware classes: For modern application processors with branch prediction, out-of-order execution, and DRAM refresh timing, producing sound WCET bounds is computationally intractable or requires unacceptable analysis pessimism.

Consequently, A RISC-V embedded cores without caches (e.g., Ibex/CV32E40P), WCET analysis is tractable. For application-class RISC-V cores (e.g., SiFive U74) with L1/L2 caches and branch predictors, it is not—a fact that restricts the applicability of certified MCS frameworks to a narrow class of hardware platforms.

1) LO-Task Starvation: As established in Section XIV-C, LO tasks receive no scheduling guarantees in HI-mode. This has three practically significant consequences:

- Unbounded starvation: A persistent or recurring overrun condition (e.g., a HI task consistently executing near $C_i(HI)$) keeps the system in HI-mode indefinitely, starving LO tasks without bound.
- No soft-degradation path: Unlike bandwidth-regulated systems (CBS, GRUB), EDF-VD has no mechanism to gradually reduce LO-task bandwidth before full suspension—it is a binary on/off transition.
- Absence of LO-task service guarantees: The formal schedulability proof guarantees LO-task deadlines in LO-mode but makes no statement about LO-task service quality in HI-mode. For systems where LO tasks include watchdog refreshes, health monitoring, or network keepalives, this silence is not acceptable.

Extensions addressing partial LO-task retention (e.g., keeping LO tasks with sufficient remaining slack) have been proposed [6] but at the cost of increased schedulability analysis complexity and runtime overhead.

Single-Core Assumption Limitations, The theoretical foundation of EDF-VD is strictly uniprocessor. Multicore MCS introduces two additional complexity layers that the single-core analysis cannot address. first Inter-core interference, On shared-memory multicore platforms, tasks on different cores compete for shared cache, memory bus, and interconnect bandwidth. This inter-core contention inflates effective WCET bounds beyond what single-core analysis predicts, invalidating HI-mode schedulability guarantees derived from single-core WCET values. Second Global vs. partitioned scheduling Multicore MCS can use partitioned scheduling (assign tasks statically to cores, apply uniprocessor MCS per core) or global scheduling (tasks migrate across cores). Partitioned MCS is simpler but wastes resources due to bin-packing inefficiency; global MCS is more efficient but has no known optimal algorithm equivalent to EDF-VD for the multicore case. The Dhall effect 7 where global EDF can fail to schedule task sets with utilization arbitrarily close to 1 applies to multicore MCS and remains unresolved in the general case.

VI. Conclusion

This paper reviewed the principles of mixed-criticality scheduling and analyzed how multiple execution-time assumptions enable safety-critical tasks to coexist with lower-criticality workloads on shared platforms. Unlike classical scheduling models based on a single WCET value, mixed-criticality systems provide mode-dependent guarantees by switching from optimistic LO-mode execution to conservative HI-mode execution when timing overruns occur. The analysis of Vestal's model and EDF-VD demonstrates that safety guarantees are achieved by prioritizing HI-criticality tasks and reducing interference from LO-criticality workloads. However, this protection introduces trade-offs, including WCET pessimism, reduced resource efficiency, and possible degradation of LO-task service. Therefore, mixed-criticality scheduling represents a balance between certification-level safety guarantees and efficient resource utilization, remaining a key research direction for future safety-critical embedded systems.

References

[1] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *J. ACM*, vol. 20, p. 46–61, Jan. 1973.

[2] H. Wang, *Principles of Integrated Airborne Avionics*. Singapore: Springer, 2021.

[3] S. Vestal, "Preemptive scheduling of multi-criticality systems with varying degrees of execution time assurance," in *28th IEEE International Real-Time Systems Symposium (RTSS 2007)*, pp. 239–243, 2007.

[4] S. K. Baruah, V. Bonifaci, G. D'Angelo, H. Li, A. Marchetti-Spaccamela, N. Megow, and L. Stougie, "Scheduling real-time mixed-criticality jobs," *IEEE Transactions on Computers*, vol. 61, no. 8, pp. 1140–1152, 2012.

[5] C. Cullmann, C. Ferdinand, G. Gebhard, D. Grund, C. Maiza, J. Reineke, and B. Triquet, "Predictability considerations in the design of multi-core embedded systems," pp. 36–42, 05 2010.

[6] S. Baruah and A. Burns, *Implementing Mixed Criticality Systems in Ada*, pp. 174–188. 06 2011.

[7] S. K. Dhall and C. L. Liu, "On a real-time scheduling problem," *Oper. Res.*, vol. 26, p. 127–140, Feb. 1978.